CSE470: Software Engineering

Agile Methodologies

Why we need Agile

1. Now a days, **the requirement of software changes frequently**, even it can change a number of times in a day.

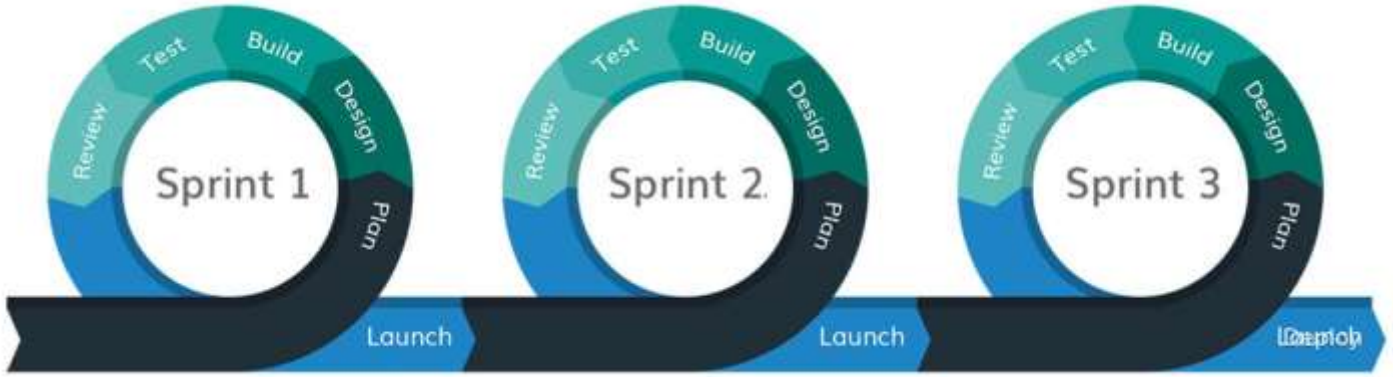
2. Agile software engineering represents a reasonable **alternative to conventional software engineering**. It has been demonstrated to deliver successful systems quickly.

3. For live systems, like Amazon, Facebook a few seconds of downtime can cause a lot. Changes in such cases needs to be smooth enough so that customers interaction do not interrupt.

Read to know more about Agile benefits: click here

What is “Agility”

- Agile is a set of **principles and values**. It is a **practice** to be followed.
- Agile is a combination of Iterative and Incremental process models with focus on **process adaptability and customer satisfaction** by **rapid delivery of working software.**
- Agile organizations tend to be **responsive rather than rigid**, which means they can better respond to change.



Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it.

✔		✘
Individuals and interactions	*over*	***processes and tools***
Working software	*over*	***comprehensive documentation***
Customer collaboration	*over*	***contract negotiation***
Responding to change	*over*	***following a plan***

Read more: https://agilemanifesto.org/
At a glance: https://agilealliance.org/

Agile Methodologies

- Extreme Programming
- Agile Unified Process (excluded from syllabus)
- Scrum

Extreme Programming

- In short also known as XP
- XP is especially valuable for projects with unclear or dynamic needs
- New versions may be built **several times per day**
- Increments are delivered to customers **every 2 weeks (approx.)**
- All tests must be run for every build and the build should be accepted by the client and if tests run successfully.

XP Workflow

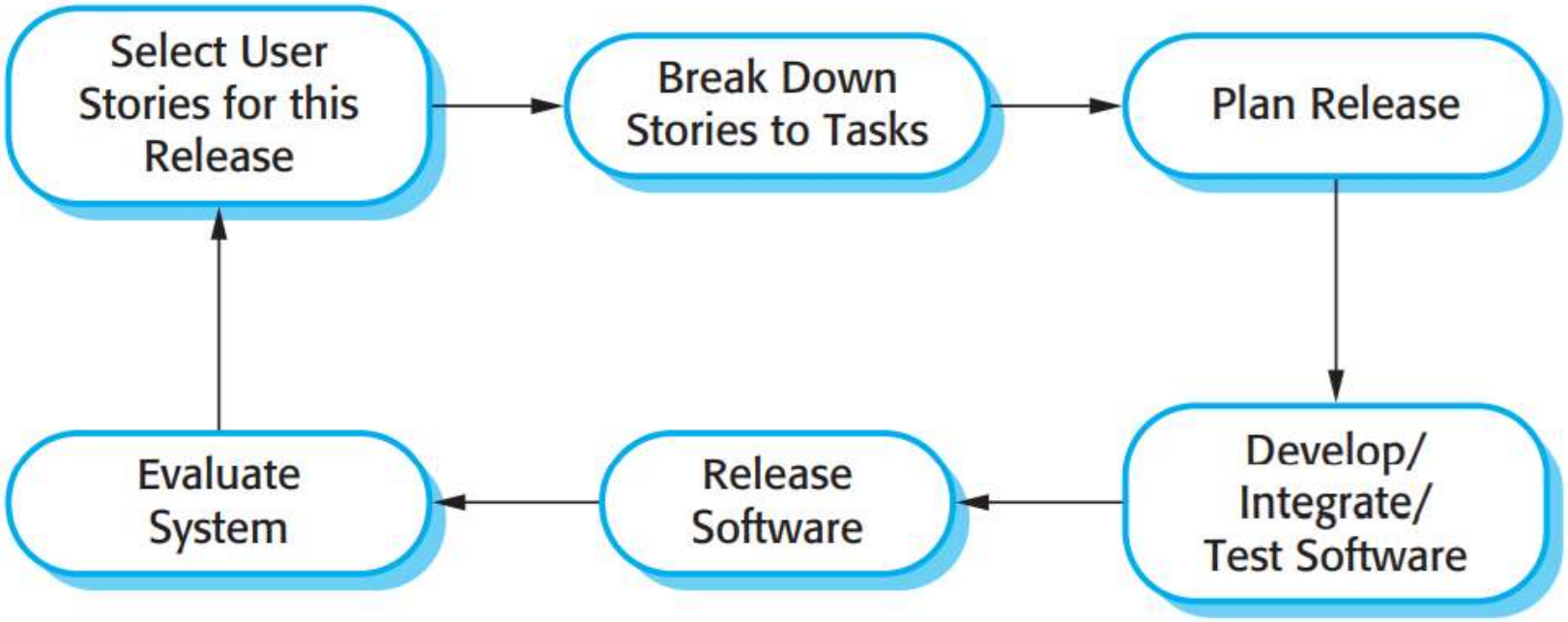


Figure 3.3 The extreme programming release cycle

Reference: Software-Engineering-9th-Edition-by-Ian-Sommerville (pdf page 82)

XP Principles or Practices

Incremental Planning	Pair Programming
Small Releases	Collective Ownership
Simple Design	Continuous Integration
Test First Development	Sustainable Development
Refactoring	On-site Customer

Pair Programming

- Two people code in share, they switch roles from observer to navigator frequently
- Observing code produces better code, so less business cost.
- Support each other, better chance for learning.

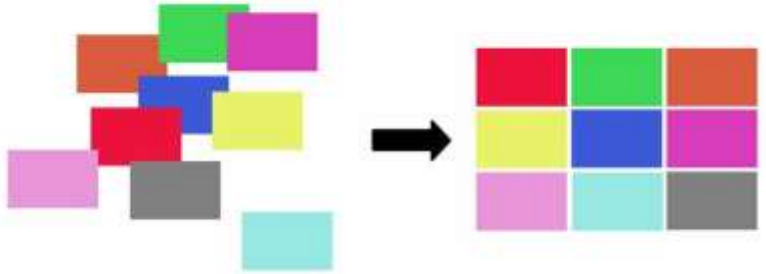
Test First Development

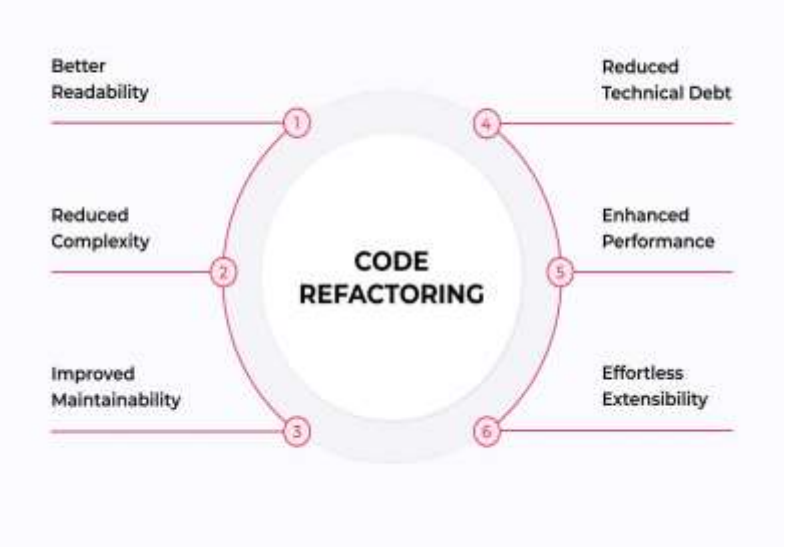
- Writes unit test for each method to identify problems and resolve it

- Test First Development (TFD) is used to ensure better code quality and less error. It refers to write test code first and after that write the development code.

Refactoring

- It’s a technique for restructuring existing code by changing its internal structure without changing its external behaviour.
- Refactoring includes reducing duplicate codes, breaking long classes/methods into small ones, appropriate variable naming and many more.
- It is done for requirement change, improving design and easy extensibility of software.



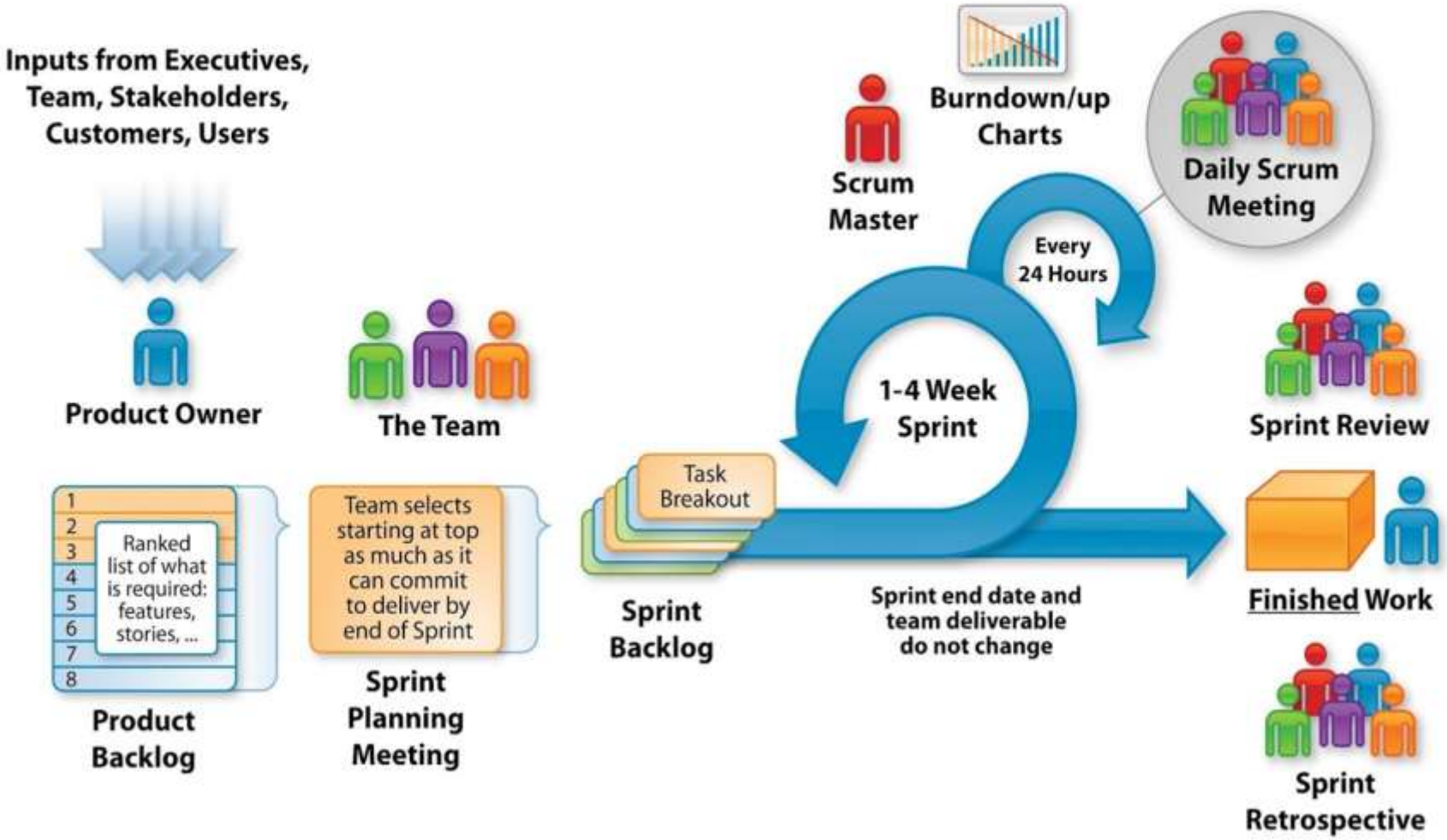


Scrum (Most popular Agile method)

- It is the most widely used Agile S/W development method for **project management**

- The full software is delivered in 1-4 weeks iterations

The Agile - Scrum Framework



Scrum Framework

Roles
Product Owner
Scrum Master
Team

Ceremonies (In Scrum, Ceremonies = Official Meetings)
Sprint Planning
Sprint Review & Retrospective
Daily Scrum Meeting

Artifacts
Product Backlog
Sprint Backlog
Burndown charts

Scrum Roles

Product Owner

- Possibly a Product Manager or Project Sponsor
- Decides features, release date, prioritization, $$$
- Conveys a clear vision to the Scrum team
- Replies to frequently asked queries

Scrum Master

- Typically a Project Manager or Team Leader
- Responsible for enacting Scrum values and practices
- Remove impediments / politics, keeps everyone productive

Project Team

- 5-10 members; Teams are self-organizing
- Cross-functional: QA, Programmers, UI Designers, etc.
- Membership should change only between sprints

Artifact - Product Backlog

- **The requirements**
- A list of all desired work on project



- Ideally expressed as a list of user stories along with "story points", such that each item has value to users or customers of the product

- Prioritized by the product owner

- Reprioritized at start of each sprint

Artifact - Product Backlog

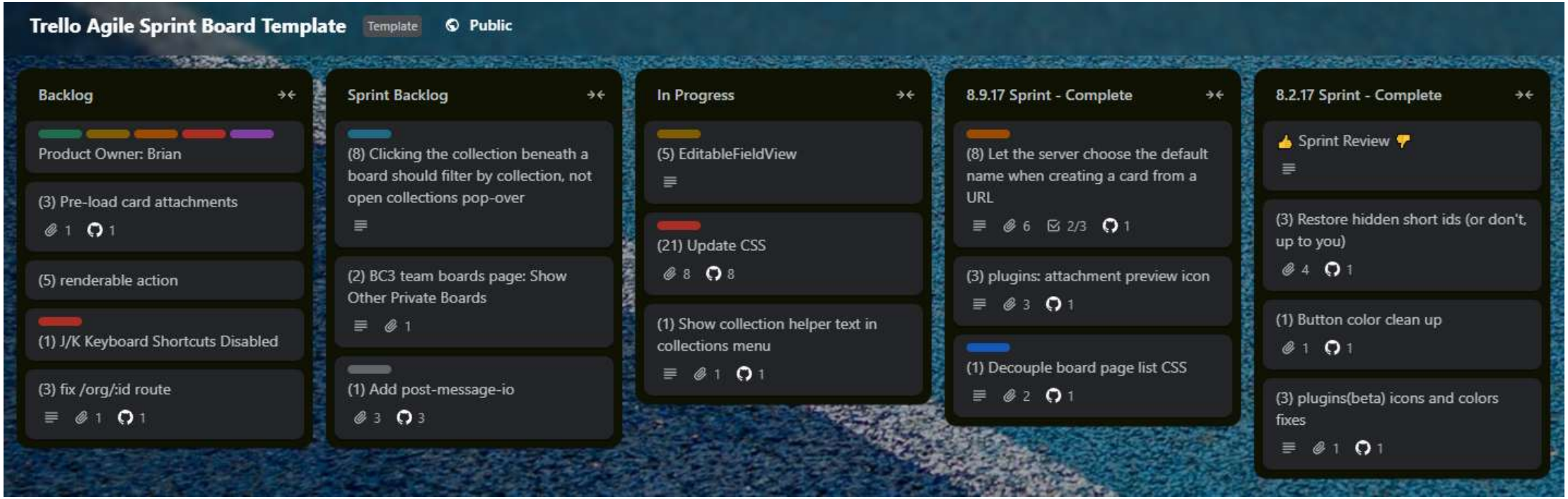
Feature	Backlog item	Estimate
A	Allow a guest to make a reservation	3 (story points)
B	As a guest, I want to cancel a reservation.	5
C	As a guest, I want to change the dates of a reservation.	3
D	As a hotel employee, I can run RevPAR reports (revenue-per-available-room)	8
E	Improve exception handling	8

Artifact - Sprint Backlog

- **High priority features** are selected based on **points from product backlog**.
- These features will be covered within that **1-4 weeks time slot** called as sprint.
- Estimated **work remaining** is updated daily
- Any team member can not **add, delete change sprint backlog**



Artifact - Sprint Board



Template Link: https://trello.com/b/ZqN99gGN

Ceremony - Daily Scrum Meeting (DSM)

Parameters

- Daily, ~15 minutes, Stand-up
- Anyone late pays a $1 fee

Not for problem solving

- Whole world is invited
- Only team members, Scrum Master, product owner, **can talk**
- **Helps avoid other unnecessary meetings**

Three questions answered by each team member:

- What did you do yesterday?
- What will you do today?
- **What obstacles are in your way?**



Artifact - Burndown Chart

A display of what work has been completed and what is left to complete **(Remaining work vs time)**

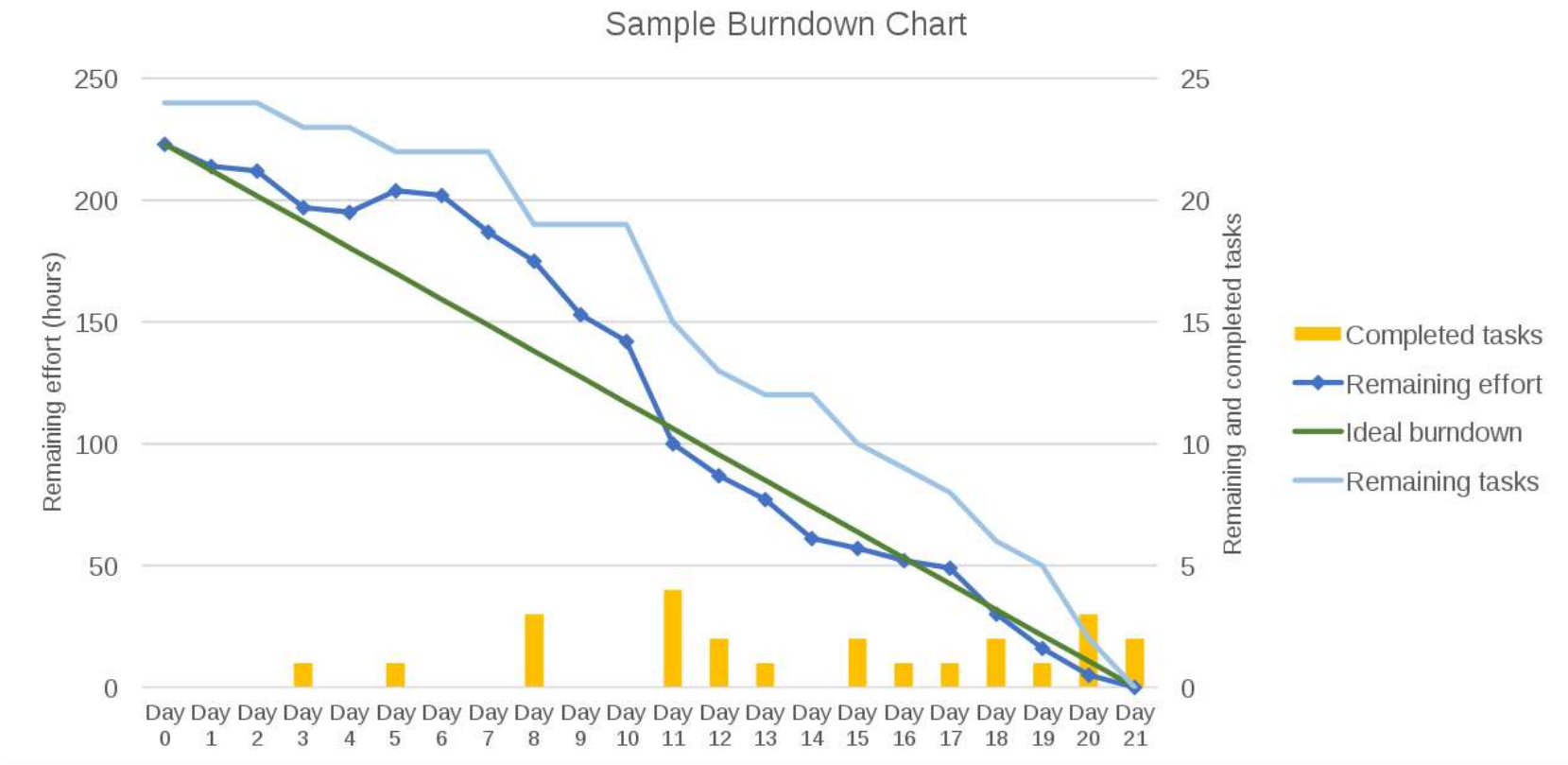
- one for each developer or work item
- updated every day
- (make best guess about **hours/points completed** each day)

Release burndown chart

- shows overall progress
- updated at end of each sprint

2 min read: https://www.mountaingoatsoftware.com/agile/scrum/scrum-tools/release-burndown

Artifact - Burndown Chart



If line goes down steadily → **Good progress**

If flat → **Team not progressing**

If goes up → **New work added (not good in sprint)**

Ceremony - Sprint Review

- Team presents **what it accomplished during the sprint**
- Demo of completed work
- Informal
  - 2-hour prep time rule
  - No slides
- Whole team participates
- Invite the world

“Show working product, not PowerPoint.”

Ceremony - Sprint Retrospective

- Identify the **scopes of improvement** for better result in next sprints
- What **worked well**, what **went wrong** are also discussed

2 min read: https://www.scrum.org/resources/what-is-a-sprint-retrospective